

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 3 | Part 1

Searching a Database

- ▶ How do we analyze the time complexity of **recursive** algorithms?
- ▶ How do we know that our recursive code is **correct**?

Databases

- Large data sets are often stored in **databases**.

PID	FullName	Level
A1843	Wan Xuegang	SR
A8293	Deveron Greer	SR
A9821	Vinod Seth	FR
A8172	Aleix Bilbao	JR
A2882	Kayden Sutton	SO
A1829	Raghu Mahanta	FR
A9772	Cui Zemin	SR
⋮	⋮	⋮

Query

- ▶ What is the name of the student with PID A8172?

Linear Search

- ▶ We *could* answer this with a linear search.
- ▶ Recall worst-case time complexity: $\Theta(n)$.
- ▶ Is there a better way?

Theoretical Lower Bounds

- ▶ **Given:** an array `arr` and a target `t`, determine the index of `t` in the array.
- ▶ Lower bound: $\Omega(n)$
 - ▶ `linear_search` has the best possible worst-case complexity!

Theoretical Lower Bounds

- ▶ **Given:** an **sorted** array `arr` and a target `t`, determine the index of `t` in the array.
- ▶ This is an **easier** problem.
- ▶ Lower bound: $\Omega(?)$

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 3 | Part 2

Binary Search

Strategy

- ▶ First pick the middle index.
- ▶ Allows you to rule out half of the other numbers.
- ▶ Pick index in the middle of what remains.
- ▶ Repeat, **recursively**.

Exercise

Fill in the blanks:

```
def binary_search(arr, t, start, stop):  
    """  
    Searches arr[start:stop] for t.  
    Assumes arr is sorted.  
    """  
    if stop - start <= 0:  
        return None  
    middle = _____ # index of the middle element  
    if arr[middle] == t:  
        return middle  
    elif arr[middle] > t:  
        return binary_search(arr, t, _____, _____)  
    else:  
        return binary_search(arr, t, _____, _____)
```

Binary Search

```
import math
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

```
import math
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

t = 21

-10	-6	-3	1	2	5	12	21	33	35	42
0	1	2	3	4	5	6	7	8	9	10

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 3 | Part 3

Thinking Inductively

Recursion

- ▶ Recursive algorithms can almost look like **magic**.
- ▶ How can we be sure that `binary_search` works?

Tips

1. Make sure algorithm works in the **base case**.
2. Check that all recursive calls are on **smaller problems**.
3. **Assuming** that the recursive calls work, does the whole algorithm work?

Base Case

- ▶ Smallest input for which you can easily see that the algorithm works.
- ▶ Recursion works by making problem smaller until base case is reached.
- ▶ Usually $n = 0$ or $n = 1$ (or even both!)

Base Case: $n = 0$

- ▶ Suppose `arr[start:stop]` is empty.
- ▶ In this case, the function returns **None**.
 - ▶ **Correct!**

Base Case: $n = 1$

- ▶ Suppose `arr[start:stop]` has one element.
- ▶ If that element is the target, the algorithm will find it.
 - ▶ **Correct!**
- ▶ If it isn't, the algorithm will recurse on a problem of size 0 and return **None**.
 - ▶ **Correct!**

Recursive Calls

- ▶ Recursive calls must be on **smaller problems**.
 - ▶ Otherwise, base case never reached. Infinite recursion!

```
import math
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

- ▶ Is arr[start:middle] smaller than arr[start:stop]?
- ▶ Is arr[middle+1:stop] smaller than arr[start:stop]?

Leap of Faith

- ▶ **Assume** the recursive calls work.
- ▶ Does the overall algorithm work, then?

Induction

- ▶ These steps can be turned into a formal proof by **induction**.
- ▶ For us, less necessary to prove to other people.
- ▶ Instead, prove to **yourself** that your code works.
- ▶ We won't be doing formal inductive proofs.

Why does this work?

- ▶ Show that it works for size 1 (base case).
- ▶ $\Rightarrow$ will work for size 2 (inductive step).
- ▶ $\Rightarrow$ will work for sizes 3, 4 (inductive step).
- ▶ $\Rightarrow$ will work for sizes 5, 6, 7, 8 (inductive step).

Exercise

Does this code work? Why or why not?

```
import math
def summation(numbers):
    n = len(numbers)
    if n == 0:
        return 0
    middle = math.floor(n / 2)
    return (
        summation(numbers[:middle])
        +
        summation(numbers[middle:])
    )
```

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 3 | Part 4

Recurrence Relations

Time Complexity of Binary Search

- ▶ What is the time complexity of `binary_search`?
- ▶ No loops!

Best Case

```
import math
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

Worst Case

Let $T(n)$ be worst case time on input of size n .

```
import math
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

Recurrence Relations

- We found

$$T(n) = \begin{cases} T(n/2) + \Theta(1), & n \geq 2 \\ \Theta(1), & n = 1 \end{cases}$$

- This is a **recurrence relation**.

Solving Recurrences

- ▶ We want simple, non-recursive formula for $T(n)$ so we can see how fast $T(n)$ grows.
 - ▶ Is it $\Theta(n)$? $\Theta(n^2)$? Something else?
- ▶ Obtaining a simple formula is called **solving** the recurrence.

Example: Getting Rich

- ▶ Suppose on day 1 of job, you are paid \$3.
- ▶ Each day thereafter, your pay is doubled.
- ▶ Let $S(n)$ be your pay on day n :

$$S(n) = \begin{cases} 2 \cdot S(n - 1), & n \geq 2 \\ 3, & n = 1 \end{cases}$$

Example: Unrolling

$$S(n) = \begin{cases} 2 \cdot S(n - 1), & n \geq 2 \\ 3, & n = 1 \end{cases}$$

- Take $n = 4$.

Solving Recurrences

We'll use a four-step process to solve recurrences:

1. "Unroll" several times to find a pattern.
2. Write general formula for k th unroll.
3. Solve for # of unrolls needed to reach base case.
4. Plug this number into general formula.

Step 1: Unroll several times

$$S(n) = \begin{cases} 2 \cdot S(n - 1), & n \geq 2 \\ 3, & n = 1 \end{cases}$$

Step 2: Find general formula

$$\begin{aligned} S(n) &= 2 \cdot S(n - 1) \\ &= 2 \cdot 2 \cdot S(n - 2) \\ &= 2 \cdot 2 \cdot 2 \cdot S(n - 3) \end{aligned}$$

On step k :

Step 3: Find step # of base case

- ▶ On step k , $S(n) = 2^k \cdot S(n - k)$.
- ▶ When do we see $S(1)$?

Step 4: Plug into general formula

- ▶ From step 2: $S(n) = 2^k \cdot S(n - k)$.
- ▶ From step 3: Base case of $S(1)$ reached when $k = n - 1$.
- ▶ So:

Solving the Recurrence

- ▶ We have **solved** the recurrence¹:

$$S(n) = 3 \cdot 2^{n-1}$$

- ▶ This is the **exact** solution. The **asymptotic** solution is $S(n) = \Theta(2^n)$.
- ▶ We'll call this method “solving by unrolling”.

¹On day 20, you'll be paid ≈ 1.5 million dollars.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 3 | Part 5

Binary Search Recurrence

Binary Search

- ▶ What is the time complexity of `binary_search`?
- ▶ Best case: $\Theta(1)$.
- ▶ Worst case:

$$T(n) = \begin{cases} T(n/2) + \Theta(1), & n \geq 2 \\ \Theta(1), & n = 1 \end{cases}$$

Simplification

- ▶ When solving, we can replace $\Theta(f(n))$ with $f(n)$:

$$T(n) = \begin{cases} T(n/2) + 1, & n \geq 2 \\ 1, & n = 1 \end{cases}$$

- ▶ As long as we state final answer using Θ notation!

Another Simplification

- ▶ When solving, we can assume n is a power of 2.

Step 1: Unroll several times

$$T(n) = \begin{cases} T(n/2) + 1, & n \geq 2 \\ 1, & n = 1 \end{cases}$$

Step 2: Find general formula

$$T(n) = T(n/2) + 1$$

$$= T(n/4) + 2$$

$$= T(n/8) + 3$$

On step k :

Step 3: Find step # of base case

- ▶ On step k , $T(n) = T(n/2^k) + k$
- ▶ When do we see $T(1)$?

Step 4: Plug into general formula

- ▶ $T(n) = T(n/2^k) + k$
- ▶ Base case of $T(1)$ reached when $k = \log_2 n$.
- ▶ So:

Note

- ▶ Remember: $\log_b x = (\log_a x)/(\log_a b)$
- ▶ So we don't write $\Theta(\log_2 n)$
- ▶ Instead, just: $\Theta(\log n)$

Time Complexity of Binary Search

- ▶ Best case: $\Theta(1)$
- ▶ Worst case: $\Theta(\log n)$

Is binary search fast?

- ▶ Suppose all 10^{19} grains of sand are assigned a unique number, sorted from least to greatest.
- ▶ Goal: find a particular grain.
- ▶ Assume one basic operation takes 1 nanosecond.
- ▶ Linear search: 317 years.
- ▶ Binary search: ≈ 60 nanoseconds.

Exercise

Binary search seems so much faster than linear search.
What's the caveat?

Caveat

- ▶ The array must be **sorted**.
- ▶ This takes $\Omega(n)$ time.

Why use binary search?

- ▶ If data is **not sorted**, sorting + binary search takes longer than linear search.
- ▶ But if doing **multiple queries**, looking for nearby elements, sort once and use binary search after.

Theoretical Lower Bounds

- ▶ A lower bound for searching a sorted list is $\Omega(\log n)$.
- ▶ This means that binary search has **optimal** worst case time complexity.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 3 | Part 6

Selection Sort and Loop Invariants

Sorting

- ▶ Sorting is a very common operation.
- ▶ But why is it important?
- ▶ Sorting makes some problems easier to solve.

Today

- ▶ How do we sort?
- ▶ How fast can we sort?
- ▶ How do we use sorted structure to write faster algorithms?

Today

- ▶ **Also:** how to understand complex loops with **loop invariants**.

Selection Sort

- ▶ Repeatedly remove smallest element.
- ▶ Put it at beginning of new list.

Example: `arr = [5, 6, 3, 2, 1]`

In-place Selection Sort

- ▶ We don't need a separate list.
 - ▶ We can swap elements until sorted.
- ▶ Store “new” list at the beginning of input list.
- ▶ Separate the old and new with a **barrier**.

Example: `arr = [5, 6, 3, 2, 1]`

```
def selection_sort(arr):  
    """In-place selection sort."""  
    n = len(arr)  
    if n <= 1:  
        return  
    for barrier_ix in range(n-1):  
        # find index of min in arr[start:]  
        min_ix = find_minimum(arr, start=barrier_ix)  
        #swap  
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix]  
        )
```

```
def find_minimum(arr, start):  
    """Finds index of minimum. Assumes non-empty."""  
    n = len(arr)  
    min_value = arr[start]  
    min_ix = start  
    for i in range(start + 1, n):  
        if arr[i] < min_value:  
            min_value = arr[i]  
            min_ix = i  
    return min_ix
```

Loop Invariants

- ▶ How we understand an iterative algorithm?
- ▶ A **loop invariant** is a statement that is true after every iteration.
 - ▶ And before the loop begins!

Loop Invariant(s)

After the α th iteration of selection sort, each of the first α elements is $\leq$ each of the remaining elements.

Example: `arr = [5, 6, 3, 2, 1]`

Loop Invariant(s)

After the α th iteration, the first α elements are sorted.

Example: `arr = [5, 6, 3, 2, 1]`

Loop Invariants

- ▶ Plug the total number of iterations into the loop invariant to learn about the result.
 - ▶ `selection_sort` makes $n - 1$ iterations:
 - ▶ After the $(n - 1)$ th iteration, the first $(n - 1)$ elements are sorted.
 - ▶ After the $(n - 1)$ th iteration, each of the first $(n - 1)$ elements is $\leq$ each of the remaining elements.

Time Complexity

```
def selection_sort(arr):  
    """In-place selection sort."""  
    n = len(arr)  
    if n <= 1:  
        return  
    for barrier_ix in range(n-1):  
        # find index of min in arr[barrier_ix:]  
        min_value = arr[barrier_ix]  
        min_ix = barrier_ix  
        for i in range(barrier_ix + 1, n):  
            if arr[i] < min_value:  
                min_value = arr[i]  
                min_ix = i  
  
        #swap  
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix]  
        )
```

Time Complexity

- ▶ Selection sort takes $\Theta(n^2)$ time.

Exercise

Modify `selection_sort` so that it computes a **median** of the input array. What is the time complexity?

```
def selection_sort(arr):  
    """In-place selection sort."""  
    n = len(arr)  
    if n <= 1:  
        return  
    for barrier_ix in range(n-1):  
        # find index of min in arr[start:]  
        min_ix = find_minimum(arr, start=barrier_ix)  
        #swap  
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix]  
        )
```

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 3 | Part 7

Mergesort

Can we sort faster?

- ▶ The tight theoretical lower bound for **comparison** sorting is $\Theta(n \log n)$.
- ▶ Selection sort is quadratic.
- ▶ How do we sort in $\Theta(n \log n)$ time?

Mergesort

- ▶ Mergesort is a fast sorting algorithm.
- ▶ Has **best possible** (worst-case) time complexity: $\Theta(n \log n)$.
- ▶ Implements **divide/conquer/recombine** strategy.

The Idea

- ▶ **Divide:** split the array into halves
 - ▶ $[6, 1, 9, 2, 4, 3] \rightarrow [6, 1, 9], [2, 4, 3]$
- ▶ **Conquer:** sort each half, recursively
 - ▶ $[6, 1, 9] \rightarrow [1, 6, 9]$ and $[2, 4, 3] \rightarrow [2, 3, 4]$
- ▶ **Combine:** merge sorted halves together
 - ▶ $[1, 6, 9], [2, 3, 4] \rightarrow [1, 2, 3, 4, 6, 9]$

Aside: splitting arrays

- ▶ Splitting an array in half by **slicing**:

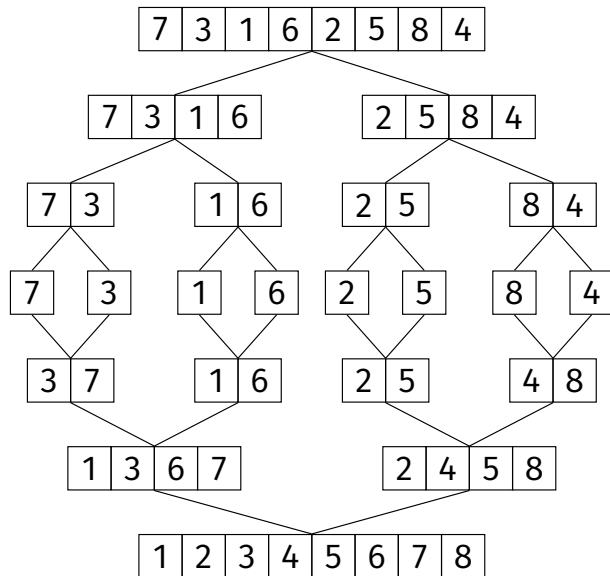
```
>>> arr = [9, 1, 4, 2, 5]
>>> middle = math.floor(len(arr) / 2)
>>> arr[:middle]
[9, 1]
>>> arr[middle:]
[4, 2, 5]
```

- ▶ **Warning!** Creates a copy!

Mergesort

```
def mergesort(arr):  
    """Sort array in-place."""  
    if len(arr) > 1:  
        middle = math.floor(len(arr) / 2)  
        left = arr[:middle]  
        right = arr[middle:]  
        mergesort(left)  
        mergesort(right)  
        merge(left, right, arr)
```

The Idea



Understanding Mergesort

1. What is the base case?
2. Are the recursive problems smaller?
3. Assuming the recursive calls work, does the whole algorithm work?

1. Base Case: $n = 1$

- ▶ Arrays of size one are trivially sorted.
- ▶ Returns immediately. **Correct!**

2. Smaller Problems?

- ▶ Are `arr[:middle]` and `arr[middle:]` always smaller than `arr`?
- ▶ Try it for `len(arr) == 2`.

3. Does it Work?

- ▶ Assume mergesort works on arrays of size $< n$.
- ▶ Does it work on arrays of size n ?

Mergesort

```
def mergesort(arr):  
    """Sort array in-place."""  
    if len(arr) > 1:  
        middle = math.floor(len(arr) / 2)  
        left = arr[:middle]  
        right = arr[middle:]  
        mergesort(left)  
        mergesort(right)  
        merge(left, right, arr)
```

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 3 | Part 8

Merge

Merging

- ▶ We have sorted each half.
- ▶ Now we need to **merge** together.

Merging

- ▶ We have sorted each half.
- ▶ Now we need to **merge** together.
- ▶ **Note:** this is an example of a problem that is made easier by sorting.

merge

3

1

merge

3

2

1

merge

3

6

1

2

merge

5

6

1

2

3

merge

7

6

1

2

3

5

merge

7

1

2

3

5

6

merge

8

1

2

3

5

6

7

merge



merge

```
def merge(left, right, out):  
    """Merge sorted arrays, store in out."""  
    left.append(float('inf'))  
    right.append(float('inf'))  
    left_ix = 0  
    right_ix = 0  
  
    for ix in range(len(out)):  
        if left[left_ix] < right[right_ix]:  
            out[ix] = left[left_ix]  
            left_ix += 1  
        else:  
            out[ix] = right[right_ix]  
            right_ix += 1
```

Loop Invariant

- ▶ Assume `left` and `right` are sorted.
- ▶ **Loop invariant:** After α th iteration,
`out[: $\alpha$ ] == sorted(left + right)[: $\alpha$ ]`

Key of mergesort

- ▶ merge is where the **actual sorting** happens.
- ▶ Example: `merge([3], [1], ...)` results in `[1,3]`

Time Complexity of merge

```
def merge(left, right, out):  
    """Merge sorted arrays, store in out."""  
    left.append(float('inf'))  
    right.append(float('inf'))  
    left_ix = 0  
    right_ix = 0  
  
    for ix in range(len(out)):  
        if left[left_ix] < right[right_ix]:  
            out[ix] = left[left_ix]  
            left_ix += 1  
        else:  
            out[ix] = right[right_ix]  
            right_ix += 1
```

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 3 | Part 9

Time Complexity of Mergesort

Time Complexity

```
def mergesort(arr):  
    """Sort array in-place."""  
    if len(arr) > 1:  
        middle = math.floor(len(arr) / 2)  
        left = arr[:middle]  
        right = arr[middle:]  
        mergesort(left)  
        mergesort(right)  
        merge(left, right, arr)
```

Aside: Copying

- ▶ What is `arr[:middle]` doing “under the hood”?
- ▶ What is the time complexity?

The Recurrence

```
def mergesort(arr):  
    """Sort array in-place."""  
    if len(arr) > 1:  
        middle = math.floor(len(arr) / 2)  
        left = arr[:middle]  
        right = arr[middle:]  
        mergesort(left)  
        mergesort(right)  
        merge(left, right, arr)
```

Solving the Recurrence

$$T(n) = 2T(n/2) + \Theta(n)$$

Solving the Recurrence

$$T(n) = 2T(n/2) + \Theta(n)$$

Optimality

- ▶ **Theorem:** Any (comparison) sorting algorithm's worst-case time complexity must be $\Omega(n \log n)$.
- ▶ Mergesort is **optimal!**

Be Careful!

- ▶ It is possible for a sorting algorithm to have a **best case** time complexity smaller than $n \log n$.
 - ▶ Insertion sort, for example.
- ▶ Mergesort has best case time complexity of $\Theta(n \log n)$.
- ▶ Mergesort is **sub-optimal** in this sense!

Be Careful!

- ▶ The $\Theta(n \log n)$ lower-bound is for **comparison sorting**.
- ▶ It is possible to sort in worst-case $\Theta(n)$ time without comparing.²

²Bucket sort, radix sort, etc.

What if?

- ▶ **Divide:** split the array into halves
- ▶ **Conquer:** sort each half **using selection sort**
- ▶ **Combine:** merge sorted halves together

mergeselectionsort

```
def mergeselectionsort(arr):  
    """Sort array in-place."""  
    if len(arr) > 1:  
        middle = math.floor(len(arr) / 2)  
        left = arr[:middle]  
        right = arr[middle:]  
        selection_sort(left)  
        selection_sort(right)  
        merge(left, right, arr)
```

Exercise

What is the time complexity of this algorithm?

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 3 | Part 10

Using Sorted Structure

Sorted structure is useful

- ▶ Some problems become **much easier** if input is sorted.
 - ▶ For example, median, minimum, maximum.
- ▶ Sorting is useful as a **preprocessing** step.

Recall: The Movie Problem

- ▶ You're on a flight that will last D minutes.
- ▶ You want to pick two movies to watch.
- ▶ You want the total time of the two movies to be **as close as possible** to D .

The Movie Problem

- ▶ Brute force algorithm: $\Theta(n^2)$
- ▶ We can do better, if movie times are **sorted**.

Example

- ▶ Flight duration $D = 155$
- ▶ Movie times: 60, 80, 90, 120, 130

	60	80	90	120	130
60					
80					
90					
120					
130					

Best pair:

The Algorithm

- ▶ Keep index of shortest and longest remaining.
- ▶ On every iteration, pair the shortest and longest.
- ▶ If this pair is too long, remove longest movie; otherwise remove shortest.
 - ▶ If times are **sorted**, finding new longest/shortest movie takes $\Theta(1)$ time!

60, 80, 90, 120, 130

The Algorithm

```
def optimize_entertainment(times, target):  
    """assume times is sorted."""  
    shortest = 0  
    longest = len(times) - 1  
  
    best_pair = (shortest, longest)  
    best_objective = None  
  
    for i in range(len(times) - 1):  
        total_time = times[shortest] + times[longest]  
  
        if abs(total_time - target) < best_objective:  
            best_objective = abs(total_time - target)  
            best_pair = (shortest, longest)  
  
        if total_time == target:  
            return (shortest, longest)  
        elif total_time < target:  
            shortest += 1  
        else: # total_time > target  
            longest -= 1  
  
    return best_pair
```

Main Idea

Sorted structure allows you to rule out possibilities without explicitly checking them. But, it requires you to spend the time sorting first.

Tip: when designing an algorithm, think about sorting the input first.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 3 | Part 11

The Median and Order Statistics

Best so far...

- ▶ Sort the list with mergesort, return middle element.
- ▶ Time complexity: $\Theta(n \log n)$.

Is sorting necessary?

- ▶ Need to sort the whole list just to find middle?
- ▶ Seems like more work than necessary.

Today

- ▶ We'll design an algorithm which runs in $\Theta(n)$ expected time.
- ▶ Much more useful than just finding median...

Order Statistics

- ▶ The median is an example of an **order statistic**.

Definition

Given n numbers, the **k th order statistic** is the k th smallest number in the collection.

Example

[99, 42, -77, -12, 101]

- ▶ 1st order statistic:
- ▶ 2nd order statistic:
- ▶ 4th order statistic:

Special Cases

- ▶ **Minimum:** 1st order statistic.
- ▶ **Maximum:** n th order statistic.
- ▶ **Median:** $\lceil n/2 \rceil$ th order statistic³.
- ▶ **p th Percentile:** $\lceil \frac{p}{100} \cdot n \rceil$ th order statistic.

³What if n is even?

Goal

- ▶ **Fast** algorithm for computing any order statistic.
- ▶ Interestingly, some seem easier than others.
- ▶ Our algorithm will find **any** order statistic in $\Theta(n)$ *expected* time.

Approach #1

- ▶ We can modify `selection_sort` to find the k th order statistic.
- ▶ Loop invariant: after k th iteration, first k elements are in final sorted order.

```
def selection_sort(arr):  
    """In-place selection sort."""  
    n = len(arr)  
    if n <= 1:  
        return  
    for barrier_ix in range(n-1):  
        # find index of min in arr[start:]  
        min_ix = find_minimum(arr, start=barrier_ix)  
        #swap  
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix]  
        )
```

```
def select_k(arr, k):  
    """Find kth order statistic."""  
    n = len(arr)  
    if n <= 1:  
        return  
    for barrier_ix in range(k):  
        # find index of min in arr[start:]  
        min_ix = find_minimum(arr, start=barrier_ix)  
        #swap  
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix]  
        )  
    return arr[k-1]
```

Exercise

What are the best case and worst case time complexities of `select_k`?

Approach #1

- ▶ 1st order statistic: $\Theta(n)$.
- ▶ n th order statistic: $\Theta(n^2)$.
- ▶ Median: $\Theta(n^2)$.
- ▶ k th order statistic: $\Theta(kn)$.

Exercise

Describe how to find any order statistic in $\Theta(n \log n)$ time.

Approach #2

- ▶ Sort with mergesort, return `arr[k-1]`
- ▶ $\Theta(n \log n)$ time. Could be better...

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 3 | Part 12

Quickselect

The Goal

- ▶ Given a collection of n numbers and an order, k .
- ▶ Find the k th smallest number in the collection.

Strategy

- ▶ Pick arbitrary index, it is our pivot, (that hasn't been ruled out).
- ▶ **Partition** the numbers around this number:
 - ▶ Move numbers smaller than this to the left,
 - ▶ Larger than this to the right.
- ▶ Let p be our pivot's new position, k be the order we want.
 - ▶ If $p = k$, return this door.
 - ▶ If $p < k$, rule out numbers to left.
 - ▶ If $p > k$, rule out numbers to right.
- ▶ Repeat.

In Code

```
import random

def quickselect(arr, k, start, stop):
    """Finds kth order statistic in numbers[start:stop]"""
    pivot_ix = random.randrange(start, stop)
    pivot_ix = partition(arr, start, stop, pivot_ix)
    pivot_order = pivot_ix + 1
    if pivot_order == k:
        return arr[pivot_ix]
    elif pivot_order < k:
        return quickselect(arr, k, pivot_ix + 1, stop)
    else:
        return quickselect(arr, k, start, pivot_ix)
```

Example

`arr = [77, 42, 11, 99, 0, 101]` `k = 3`

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 3 | Part 13

Partition

Partitioning

- ▶ Given an array of n numbers and the index of a **pivot** p .
- ▶ Rearrange elements so that:
 - ▶ Everything $< p$ is first.
 - ▶ Everything $= p$ is next.
 - ▶ Everything $> p$ is last.
- ▶ Return index of first element $\geq p$.

```
def partition(arr, start, stop, pivot_ix):  
    """Partition arr[start:stop] around pivot."""  
    left = []  
    pivot_count = 0  
    right = []  
    pivot = arr[pivot_ix]  
    for ix in range(start, stop):  
        if arr[ix] < pivot:  
            left.append(arr[ix])  
        elif arr[ix] == pivot:  
            pivot_count += 1  
        else:  
            right.append(arr[ix])  
    ix = start  
    for x in left:  
        arr[ix] = x  
        ix += 1  
    for i in range(pivot_count):  
        arr[ix] = pivot  
        ix += 1  
    for x in right:  
        arr[ix] = x  
        ix += 1  
    return start + len(left)
```

Partition

- ▶ partition takes $\Theta(n)$ time.
 - ▶ This is **optimal**.
- ▶ But we can use memory more efficiently.

Motivation

- ▶ Similar to selection sort, we'll use **two** barriers:
- ▶ “Middle” barrier:
 - ▶ Separates things $<$ pivot from things $\geq$
 - ▶ Points to first thing in “right”
- ▶ “End” barrier:
 - ▶ Separates processed from unprocessed.
 - ▶ Points to first unprocessed thing.

Example

Simplification: start by moving pivot to end.

```
arr = [77, 42, 11, 99, 0, 101]    pivot = 1
```

```
def in_place_partition(arr, start, stop, pivot_ix):
    def swap(ix_1, ix_2):
        arr[ix_1], arr[ix_2] = arr[ix_2], arr[ix_1]

    pivot = arr[pivot_ix]
    swap(pivot_ix, stop-1)
    middle_barrier = start
    for end_barrier in range(start, stop - 1):
        if arr[end_barrier] < pivot:
            swap(middle_barrier, end_barrier)
            middle_barrier += 1
        # else:
            # do nothing
    swap(middle_barrier, stop-1)
    return middle_barrier
```

Efficiency

- ▶ Also takes $\Theta(n)$ time.
- ▶ No auxiliary memory required.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 3 | Part 14

Time Complexity Analysis

Time Complexity

- ▶ What is time complexity of quickselect?

```
import random
def quickselect(arr, k, start, stop):
    """Finds kth order statistic in numbers[start:stop]"""
    pivot_ix = random.randrange(start, stop)
    pivot_ix = partition(arr, start, stop, pivot_ix)
    pivot_order = pivot_ix + 1
    if pivot_order == k:
        return arr[pivot_ix]
    elif pivot_order < k:
        return quickselect(arr, k, pivot_ix + 1, stop)
    else:
```

Problem

- ▶ We don't know the size of the subproblem.
 - ▶ Is random, can be anywhere from 1 to $n - 1$.
- ▶ Difficult to write recurrence relation.

Good and Bad Pivots

- ▶ Some pivots are better than others.
 - ▶ **Good**: splits array into roughly balanced halves.
 - ▶ **Bad**: splits array into wildly unbalanced pieces.

Exercise

Suppose we're searching for the minimum. What would be the worst possible pivot?

Worst Case

- ▶ Suppose we're searching for $k = 1$ (minimum).
- ▶ Worst pivot: the maximum.
- ▶ Worst case: use max as pivot every time.
- ▶ Subproblem size: $n - 1$.

Worst Case

- ▶ Every recursive call is on problem of size $n - 1$.
- ▶ $T(n) = T(n - 1) + \Theta(n)$.
 - ▶ Solution: $\Theta(n^2)$.
- ▶ Intuitively, randomly choosing largest number as pivot every time is **very** unlikely!

$$\frac{1}{n} \times \frac{1}{n-1} \times \frac{1}{n-2} \times \dots \times \frac{1}{3} \times \frac{1}{2} = \frac{1}{n!}$$

Equally Unlikely

- ▶ Pivot falls exactly in the middle, every time.
- ▶ Subproblems are of size $n/2$.
- ▶ $T(n) = T(n/2) + \Theta(n)$.
 - ▶ Solution: $\Theta(n)$.

Typically

- ▶ Pivot falls somewhere in the middle.
- ▶ Sometimes **good**, sometimes **bad**.
- ▶ But **good** pivots reduce problem size by **so much** that they make up for **bad** pivots.

Analogy

- ▶ You're 100 miles away from home.
- ▶ You have a button that, if you press it, teleports you **1 mile** closer to home.
- ▶ How many times must you press it before you're 1 mile away from home?

Analogy

- ▶ You're 100 miles away from home.
- ▶ You have a button that, if you press it, teleports you **half the distance** to home.
- ▶ How many times must you press it before you're 1 mile away from home?

Analogy

- ▶ You're 100 miles away from home.
- ▶ You have a button that, if you press it, teleports you **half the distance** to home with probability $1/2$, does nothing with probability $1/2$.
- ▶ How many times must you press it before you're 1 mile away from home?

Analysis

- ▶ We'll call a pivot **good** if it falls in $[\frac{n}{4}, \frac{3n}{4}]$.
 - ▶ Probability: $1/2$
 - ▶ Max problem size: $3n/4$.
- ▶ We'll call a pivot **bad** if it falls outside $[\frac{n}{4}, \frac{3n}{4}]$.
 - ▶ Probability: $1/2$
 - ▶ Max problem size: $n - 1$.

Observation

$T(n)$ = time to get from n to base case

Observation

$$\begin{aligned} T(n) = & \text{time to get from } n \text{ to } \frac{3}{4}n \\ & + \text{time to get from } \frac{3}{4}n \text{ to } \left(\frac{3}{4}\right)^2 n \\ & + \text{time to get from } \left(\frac{3}{4}\right)^2 n \text{ to } \left(\frac{3}{4}\right)^3 n \\ & + \dots \end{aligned}$$

Observation

Expected $T(n)$ = expected time to get from n to $\frac{3}{4}n$
+ expected time to get from $\frac{3}{4}n$ to $\left(\frac{3}{4}\right)^2 n$
+ expected time to get from $\left(\frac{3}{4}\right)^2 n$ to $\left(\frac{3}{4}\right)^3 n$
+ ...

Related

- ▶ What is the expected number of coin flips necessary in order to see “heads”?

Related

- ▶ What is the expected number of coin flips necessary in order to see “heads”?
- ▶ Answer: 2

Implication

- ▶ Expected number of calls necessary to go from n to $3n/4$ is two.
- ▶ First call does cn work, second does $c \times (3/4)n$, third does $c \times (3/4)^2 n$, ...

Observation

Expected $T(n)$ = expected time to get from n to $\frac{3}{4}n$
+ expected time to get from $\frac{3}{4}n$ to $\left(\frac{3}{4}\right)^2 n$
+ expected time to get from $\left(\frac{3}{4}\right)^2 n$ to $\left(\frac{3}{4}\right)^3 n$
+ ...

Total Expected Time

$$2cn + 2\left(\frac{3}{4}\right)cn + 2\left(\frac{3}{4}\right)^2 cn + \dots = 2cn \cdot \sum_{p=0}^{\infty} \left(\frac{3}{4}\right)^p$$

Quickselect

- ▶ Expected time complexity: $\Theta(n)$.
- ▶ Worst case: $\Theta(n^2)$, but **very unlikely**.

Median

- ▶ We can find the median in expected linear time with **quickselect**.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 3 | Part 15

Quicksort

mergesort

- ▶ We saw mergesort.
- ▶ **Divide:** split list directly down the middle
- ▶ **Conquer:** sort each half
- ▶ **Combine:** merge sorted halves together

merge does all the work

- ▶ In mergesort, we are lazy when we divide.
- ▶ So we have to work to combine.

$[4, 1, 3, 2] \rightarrow [4, 1], [3, 2] \rightarrow [4, 3], [2, 3] \rightarrow [1, 2, 3, 4]$

What if?

- ▶ Suppose we divide so that everything in left is smaller than everything in right:
- ▶ After sorting, no need for merge.
- ▶ $[5, 1, 3, 8, 6, 2] \rightarrow [1, 3, 2], [5, 8, 6]$

What if?

- ▶ Suppose we divide so that everything in left is smaller than everything in right:
- ▶ After sorting, no need for merge.
- ▶ $[5, 1, 3, 8, 6, 2] \rightarrow [1, 3, 2], [5, 8, 6]$
- ▶ This is what partition does!

Quicksort

```
def quicksort(arr, start, stop):  
    """Sort arr[start:stop] in-place."""  
    if stop - start > 1:  
        pivot_ix = random.randrange(start, stop)  
        pivot_ix = partition(arr, start, stop, pivot_ix)  
        quicksort(arr, start, pivot_ix)  
        quicksort(arr, pivot_ix+1, stop)
```

Time Complexity

- ▶ Average case: $\Theta(n \log n)$
- ▶ Worst case: $\Theta(n^2)$.
- ▶ Like with quickselect, worst case is **very rare**.

Mergesort vs Quicksort

- ▶ Mergesort has better worst case complexity.
- ▶ But in practice, Quicksort is often faster.
- ▶ Takes less memory, too.

Memory Requirements

- ▶ merges requires output array, $\Theta(n)$ additional space.
- ▶ partition works in-place, requires no additional space⁴
- ▶ Example: sorting 3 GB of data with 4 GB of RAM.

⁴Call stack for quicksort requires $\Theta(\log n)$ additional space.